発展プログラミング (Advanced Programming)

1. オブジェクト指向プログラミングの復習

尾崎 知伸 ＜ozaki.tomonobu@nihon-u.ac.jp＞

オリエンテーション：本講義について

テーマ：オブジェクト指向とデザインパターン

- 逐次実行・条件分岐・繰り返しがあれば，すべてのアルゴリズムを構成できる
  - 1年次のProcessingや，2年次のC言語で学んだ通りです
  - クラス構成等を考えなくても，「頑張れば」何でも作ることは可能
  - しかし，保守性・再利用性・拡張性が低くなる
    - 毎回 ゼロから作り直しになる．．．
- 高い保守性・再利用性・拡張性を持つクラス・インタフェースの構成は？
- どの様なクラス構成・インタフェースの構成が望ましいのか？
  - 各クラスは，どの様な機能を持つべきか？
  - クラスの継承関係は？
  - インタフェースの継承関係は？
  - どのクラスでどのインタフェースを実装するのか？　などなど
- 「デザインパターン」（see Wikipedia）
  - 典型的な状況（パターン）における，クラス・インタフェースの構成に関する過去の知見を集約したもの
  - 万能ではないが，複雑なシステムを作成する際の一つの指針となる
- デザインパターンを学ぶ主目的
  - 再利用しやすいプログラム構成（クラス・インタフェースの構成）の修得

前提条件と資料・参考文献について（01〜05回）

- 以下の2講義の内容を理解していることを前提とする
  - 「オブジェクト指向プログラミング入門」（2年前期）
  - 「オブジェクト指向プログラミング」（3年前期）

- 資料について（いずれも受講生に対する限定公開）
  - 原則，講義資料（PDFファイル）と基本となるプログラムを配布する

- 参考書（下記書籍の内容を中心に講義・演習を行う）
  - 結城浩『Java言語で学ぶデザインパターン入門第３版』 SBクリエイティブ(2021)

- 参考資料
  - 平澤章 『オブジェクト指向でなぜつくるのか 第2版』日経BP(2011)
  - 結城浩 『Java言語プログラミングレッスン 第3版(下) オブジェクト指向を始めよう』SBクリエイティブ(2012)
  - 結城浩 『Java言語で学ぶリファクタリング入門』SBクリエイティブ(2007)
  - 高橋麻奈 『やさしいJava オブジェクト指向編 』SBクリエイティブ(2012)

進め方と課題について（01～05回）

- 演習に用いる言語：Java言語
- 演習環境：特に指定しません
  - （OS含め）独自のJava環境の利用を禁止しない（ご自身で利用しやすい環境でOK）

dockerはこちら

```
$docker  run  -it  --rm  -v  .:/home/is-user/work/  is54chs/java
```

- 課題１：各回の課題
  - 01～05回の各回それぞれで課題を出題する
  - 授業終了の5分後までに一度提出する
    - （出席と合わせ）参画度として評価（原則，努力の跡があればOK）：成績の3.3%に反映
  - 授業終了の96時間後（＝月曜日10:30:00）まで，再提出可能とする
    - レポートとしての評価：成績の 10% (＝ 2% * 5回)に反映
      - （プログラムは簡単なので？）主に「記述部」を重視して採点します

科目全体では
・参画度 10%
・レポート 30% x 3

- 課題２：デザインパターンの最終課題
  - 05回目に出題予定（締切は出題後，3週間程度を予定）
  - 内容：デザインパターンの実践
    - デザインパターンの利点・効果・必要性などの考察・説明
    - 任意のデザインパターンを用いたリファクタリングとその効果の説明　などなど
  - 成績の20%に反映する（それなりの質を求めます）
  - 未提出の場合は単位不可（E判定）

講義予定（あくまで予定です）

09/24	01. オブジェクト指向プログラミングの復習 ・抽象クラス ・インタフェース ・UML／総称型（ジェネリクス）
10/01	02. デザインパターンの導入 ・Template Methodパターン ・Adapter パターン ・Iterator パターン
10/08	03. デザインパターンの基礎 → 生成に関するパターン ・Singleton パターン ・Builder パターン ・Factory Method パターン
10/15	04. デザインパターンの応用 → 構造に関するパターン ・Bridge パターン ・Composite パターン ・Decorator パターン
10/22	05. デザインパターンの実践 → 振る舞いに関するパターン ・Visitor パターン ・Chain of Responsibility パターン ・State パターン

オブジェクト指向プログラミングとJava言語

オブジェクト指向プログラミング

キーワード
再利用性・部品化・疎結合

- 「オブジェクト指向プログラミング」
  - 「部品」のプログラミング
  - メッセージパッシング（部品間の連携）

→ 本質の一つは プログラムの「**部品化**」

- 部品化の利点：**再利用性**と**保守性**（物理的な部品の例としてUSBメモリを想像してみよう）
  - 同じ「部品」を使い回す（色々な場面で利用する）ことができる
  - 内部の処理は気にすることなく，利用することができる
  - 部品を利用する側は，（同じインタフェースを持つことを条件に）種々の部品を利用したり，機能拡張することができる
    - このとき，利用する側の変更は不要である
- 再利用性・保守性を高めるために必要なこと：部品間での”規格の統一”と”疎な結合”
  - 部品内で必要な処理は部品内で行う（独立性）
  - ある部品を作るのに，別の部品を（再）利用する（継承や委譲など）
  - 内部処理を隠蔽（カプセル化など）し，使い方（規格，メソッド）のみを公開する
  - 複数の部品で「使い方」を統一する（抽象メソッド，インタフェースなど）
  - ある部品の変更が他に影響しないようにする（カプセル化など）
- メッセージパッシング　（部品の連携）
  - 部品間の連携：部品を動かす＝メソッドを実行する
  - メッセージパッシング= メソッドの呼び出し と 結果の受け取り
  - 規格（メソッド）が同じなら，相手を気にせず呼び出せる．
  - 呼び出し側は，具体的な内部処理は気にしない．呼び出された側の責任で実行

三大特徴
1. 継承
2. カプセル化
3. 多態性

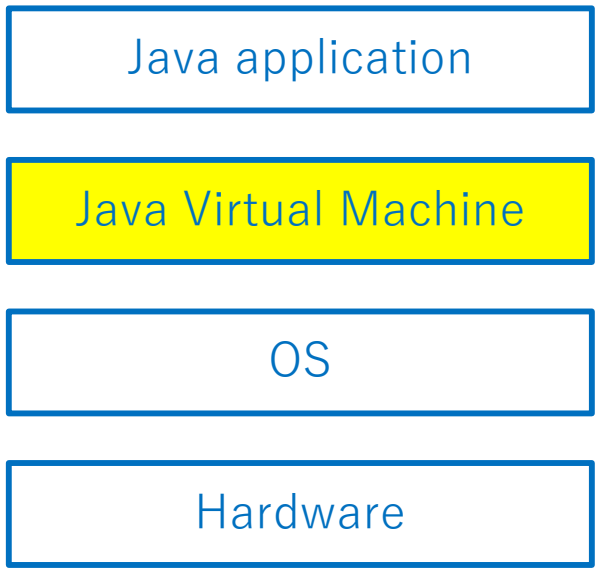
Java言語

- 代表的なオブジェクト指向プログラミング言語のひとつ

- WORA：Write Once, Run Anywhere
  - コンパイルされたJavaコードは，再コンパイルせずに他のプラットフォームで動作する
  - コンパイル
    - ソースコード（.javaファイル）→ Javaバイトコード（.classファイル）

- JVM：Java Virtual Machine
  - Javaバイトコード（Javaソースファイルをコンパイルしたもの）を実行する環境
  - JVMがOS，ハードウェアの差を吸収するため，WORAが実現できる

- Jan 2026 RedMonk プログラミング言語ランキング
  - Githubのプロジェクト数・ Stack Overflowのタグ出現回数
  - https://redmonk.com/sogrady/2026/04/14/language-rankings-1-26/
  - 第１位：JavaScript
  - 第２位：Python
  - 第３位：Java
  - 第４位：PHP
  - 第４位：C#



基本的な用語

- 振り返りのために，用語とその概要を列挙します．各自で確認・復習してください

- クラス：部品の設計図，構造体＋メソッド
  - インスタンス：実体（クラス＝設計図から生成されたもの）
  - フィールド変数：各インスタンスが持つ変数
  - メソッド：インスタンスが持つ（自身に関する）操作
- 継承：特性を引き継ぐこと．クラス間（とインタフェース間）のis_a関係を定義する
  - サブクラス / スーパークラス：継承先のクラス / 継承元のクラス
  - オーバーライド：継承先で，メソッドの処理内容を上書きすること
  - オーバーロード：引数の型・個数の異なる同じ名前のメソッドを定義すること
- 抽象クラス：抽象メソッドを持つクラス
  - 抽象メソッド：規約（意味，メソッド名，引数，返値など）だけ定義され，実装，実体を持たないメソッド
- インタフェース：
  - 抽象メソッド（と定数）の集合．メソッド構成に対する名前（目印）
  - 該当クラスが備えているべきメソッド（has_a関係）を定義するもの
  - クラス階層（is_a関係）とは異なる「型」（種別）を与える
- カプセル化：クラス外からのアクセスを制御する仕組み
- 多態性：
  - オブジェクトが複数の型に属することを許す性質
  - メソッドの動作が，呼び出されたオブジェクトに依存して決まる性質

抽象クラスとインタフェース

具体例と共に，以降のスライドで振り返ります

- 抽象メソッド：メソッドの規約（型，引数，意味）だけを定義したメソッド
- メソッドの規約？？中身がない？
  - 例：図形を表すオブジェクトが，（自身の）面積を返すメソッド
    - abstract float getArea( );
    - 「面積を返す」というメソッドに意味は決まっているが，具体的な図形が決まらないと，計算方法が決まらず，実装できない
- 抽象クラス：抽象メソッドを含むクラス
  - 抽象クラスのインスタンスを生成することはできない
  - 抽象クラスを継承した（具体）クラスは，抽象メソッドを実装しなければならない → オーバーライド（メソッドの上書き）忘れの防止
- インタフェース
  - 抽象メソッド（と定数）の集合．メソッド構成に対する名前（目印）
  - 該当クラスが備えているべきメソッド（has_a関係）を定義するもの
  - クラス階層（is_a関係）とは異なる「型」（種別）を与える
  - 多重継承の実現
- インタフェースを実装したクラスの宣言
  - class クラス名 implements インタフェース名1,インタフェース名2{ }
  - インタフェース＝抽象メソッドの集合 →（具体）クラスは，抽象メソッドを実装する必要がある

主な修飾子

アクセス修飾子

- クラス，フィールド（変数），メソッドのアクセス制限（＝公開範囲）を示す修飾子

- public
  - すべてのクラスからアクセス可能
- 修飾子なし（パッケージプライベート）
  - 同じパッケージ内と現在のクラスからアクセス可能
- protected
  - 該当クラスを継承したサブクラスからアクセス可能
- private
  - 同じクラスからのみアクセス可能

その他の修飾子の例

- abstract
  - 抽象クラス，抽象メソッドに指定
- static
  - クラスメソッド，クラス変数に指定
- final
  - 再代入・オーバーライドを禁止

パッケージ

- パッケージ
  - Javaのクラス・インタフェースを分類するための仕組み
  - アクセス制御の単位としても利用される

- Javaにおけるパッケージの実体は「ディレクトリ」
  - ディレクトリを使って，プログラム(.javaファイル）を整理するイメージ
  - サンプルプログラムは，適当なディレクトリに分けて配布する
    - 本来は，機能などのまとまりを考慮してパッケージを作成する

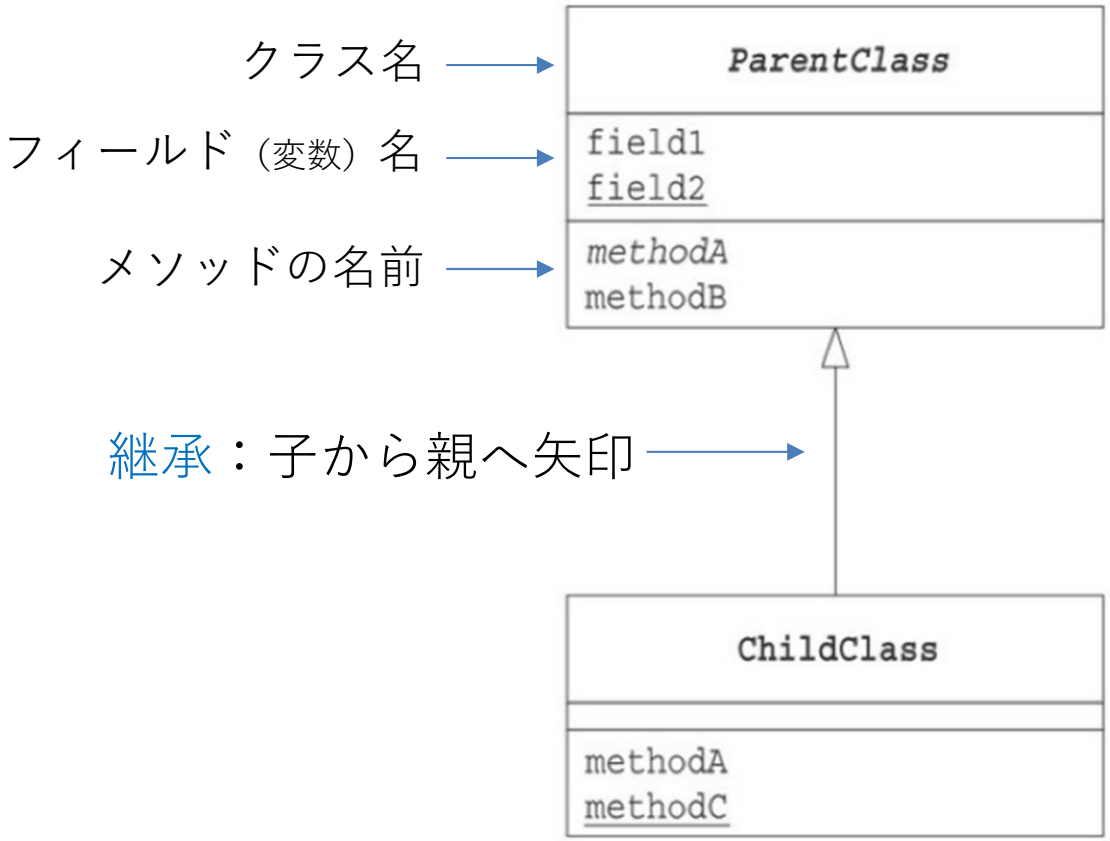
- package宣言
  - パッケージ内のプログラム（.javaファイル）の先頭に，package宣言が必要
  - デリミタは”.”
    - package lec01;　　// ディレクトリ lec01中の.javaファイル
    - package lec01.hoge;　//ディレクトリlec01/hoge中の.javaファイル

- コンパイルと実行：相対パスで指定する
  - javac lec01/hoge/xxx.java
  - java lec01/hoge/xxx

UMLとクラス図

UMLとクラス図

- UML（Unified Modeling Language）
  - システムを視覚化したり，仕様や設計を文書化したりするための表現方法の一つ
  - この授業では，Javaのクラス間の関係等を示すために利用する
- クラス図（Class Diagram）
  - クラスやインスタンス，インタフェースなどの「静的」な関係を表現したもの




```
abstract class ParentClass {
    int field1;
    static char field2;
    abstract void methodA();
    double methodB() {
        // ...
    }
}
```

```
class ChildClass extends ParentClass {
    void methodA() {
        // ...
    }
    static void methodC() {
        // ...
    }
}
```

抽象｛クラス，メソッド｝は斜体
static｛フィールド，メソッド｝は下線

結城浩 『Java言語で学ぶデザインパターン入門第３版』 SBクリエイティブ(2021)より

アクセス制御
- : private
: protected
+ :public
~ : 同一パッケージ内のみ




```
class Something {
    private   int privateField;
    protected int protectedField;
    public    int publicField;
    int packageField;
    private   void privateMethod() {
    }
    protected void protectedMethod() {
    }
    public    void publicMethod() {
    }
    void packageMethod() {
    }
}
```

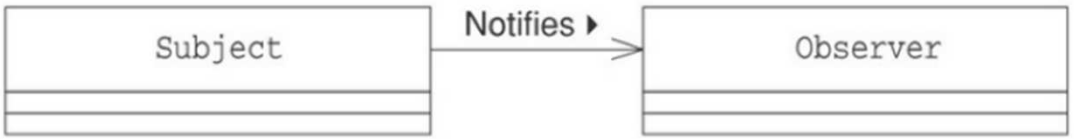
クラスの関連：
クラスの関連を表すために，関連の名前に黒三角を付けることができる



ClientがTargetを使用する



FactoryがProcutを生成する



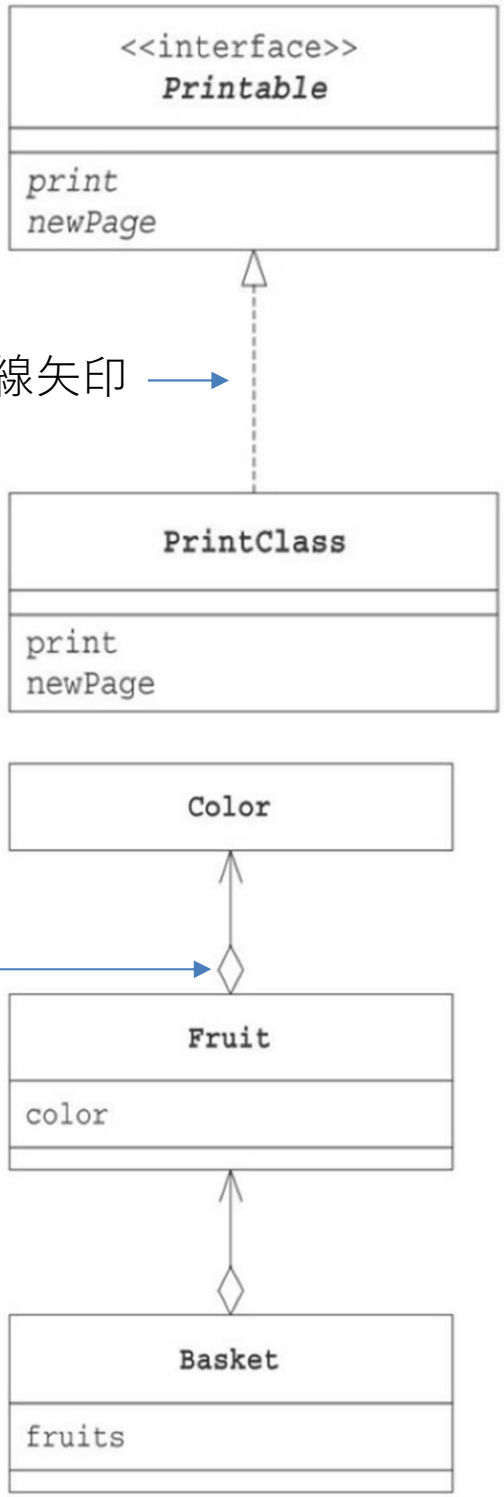
SubjectがObserverに通知する

結城浩 『Java言語で学ぶデザインパターン入門第３版』 SBクリエイティブ(2021)より

インタフェース名は斜体

実装：実装元からの点線矢印

集約（aggregation）
「持っている」関係
白抜きひし形付き矢印




```
interface Printable {
    abstract void print();
    abstract void newPage();
}

class PrintClass implements Printable {
    void print() {
        // ...
    }
    void newPage() {
        // ...
    }
}
```

```
class Color {
    // ...
}

class Fruit {
    Color color;
    // ...
}

class Basket {
    Fruit[] fruits;
    // ...
}
```

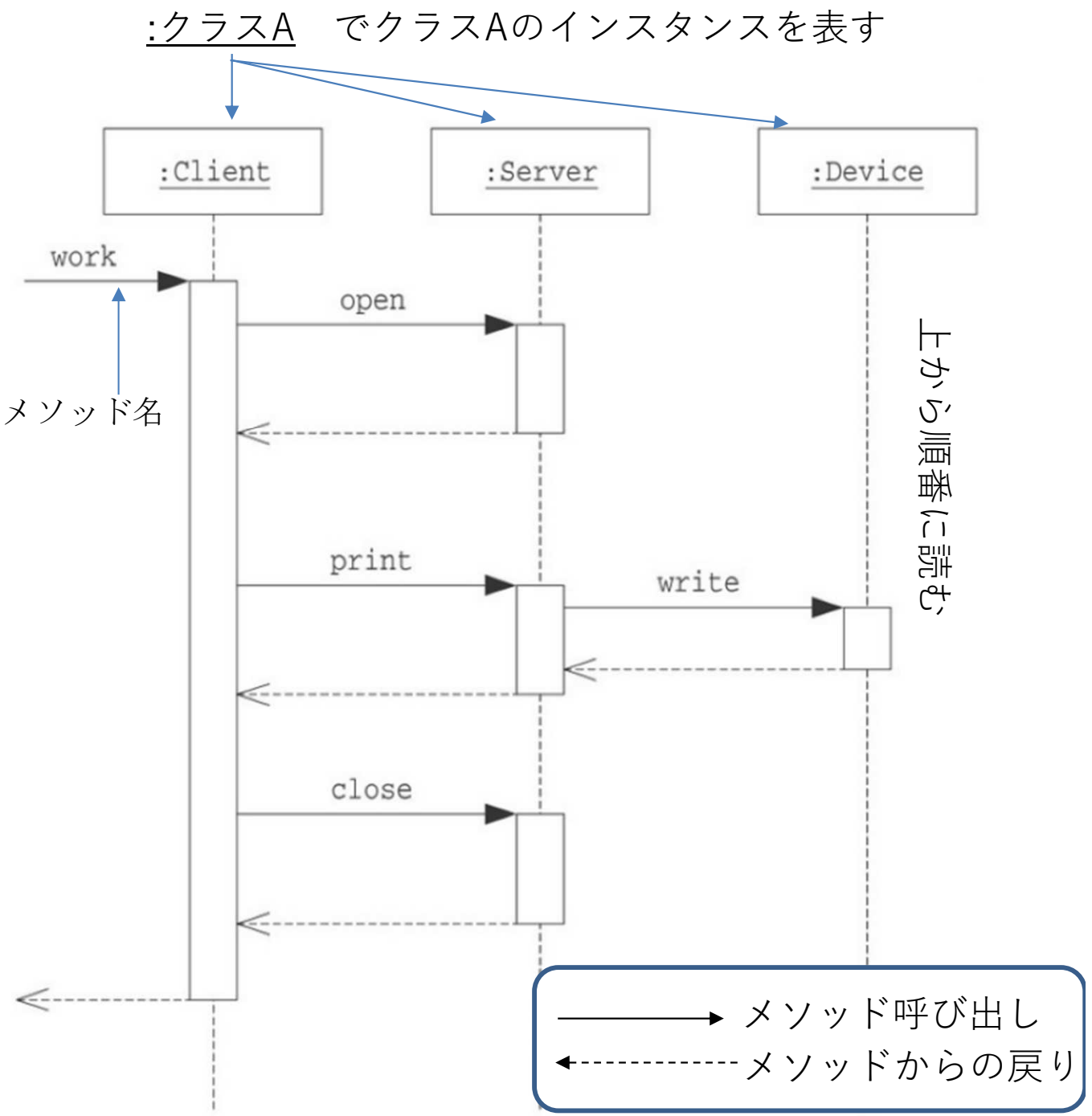
結城浩 『Java言語で学ぶデザインパターン入門第３版』 SBクリエイティブ(2021)より

シーケンス図：処理の流れとオブジェクト間の協調動作

```
class Client {
    Server server;
    void work() {
        server.open();
        server.print("Hello");
        server.close();
    }
    // ...
}

class Server {
    Device device;
    void open() {
        // ...
    }
    void print(String s) {
        device.write(s);
        // ...
    }
    void close() {
        // ...
    }
    // ...
}

class Device {
    void write(String s) {
        // ...
    }
}
```



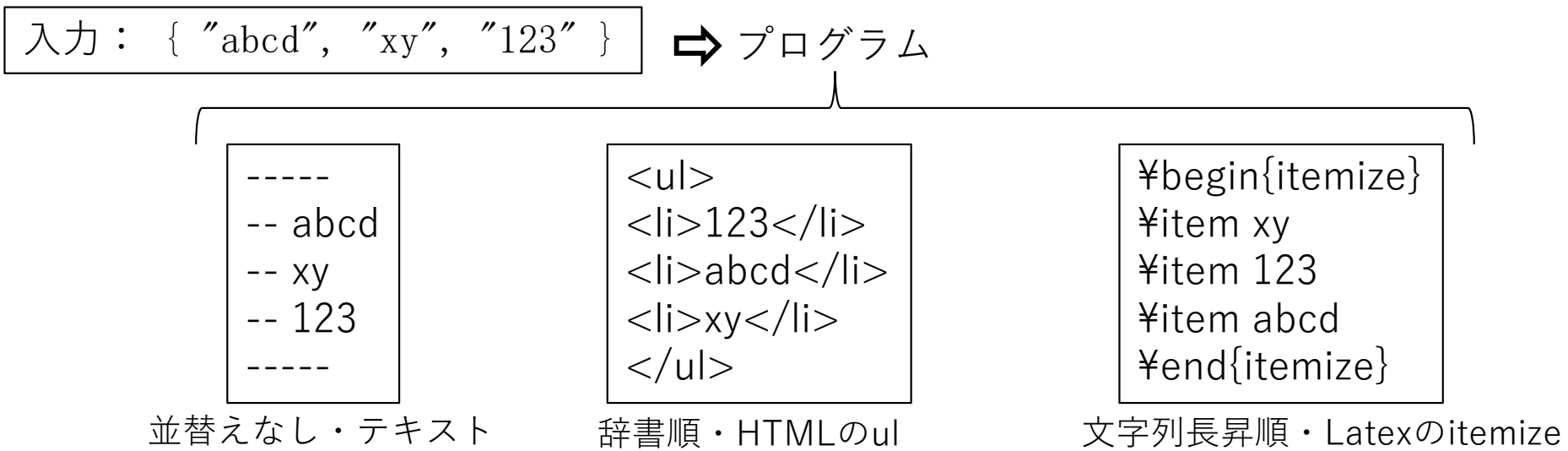
結城浩 『Java言語で学ぶデザインパターン入門第３版』 SBクリエイティブ(2021)より

抽象クラスとインタフェースの利用例
～ リスト要素の整列と表示 ～

リストの整列・表示プログラム

- 以下の例題を通じて，（抽象クラス・インタフェース等を利用した）オブジェクト指向らしい？プログラムを考える
- 実現したい内容

  配列に格納された（複数の）文字列を，
  (1) 順番を変えず，もしくは，指定された基準 で並べ替えた上で
  (2) 標準出力，または，指定されたファイル に
  (3) 指定された形式 で出力する
- 並べ替えの基準：辞書順 や 文字列長の昇順 （後から基準を増やしたい）
- リストの形式：{ テキスト， HTMLのul，Latexのitemize環境 }
  - 後から形式を増やしたい（HTMLのolやLatexのenumerateなど）



リストの整列・表示プログラム

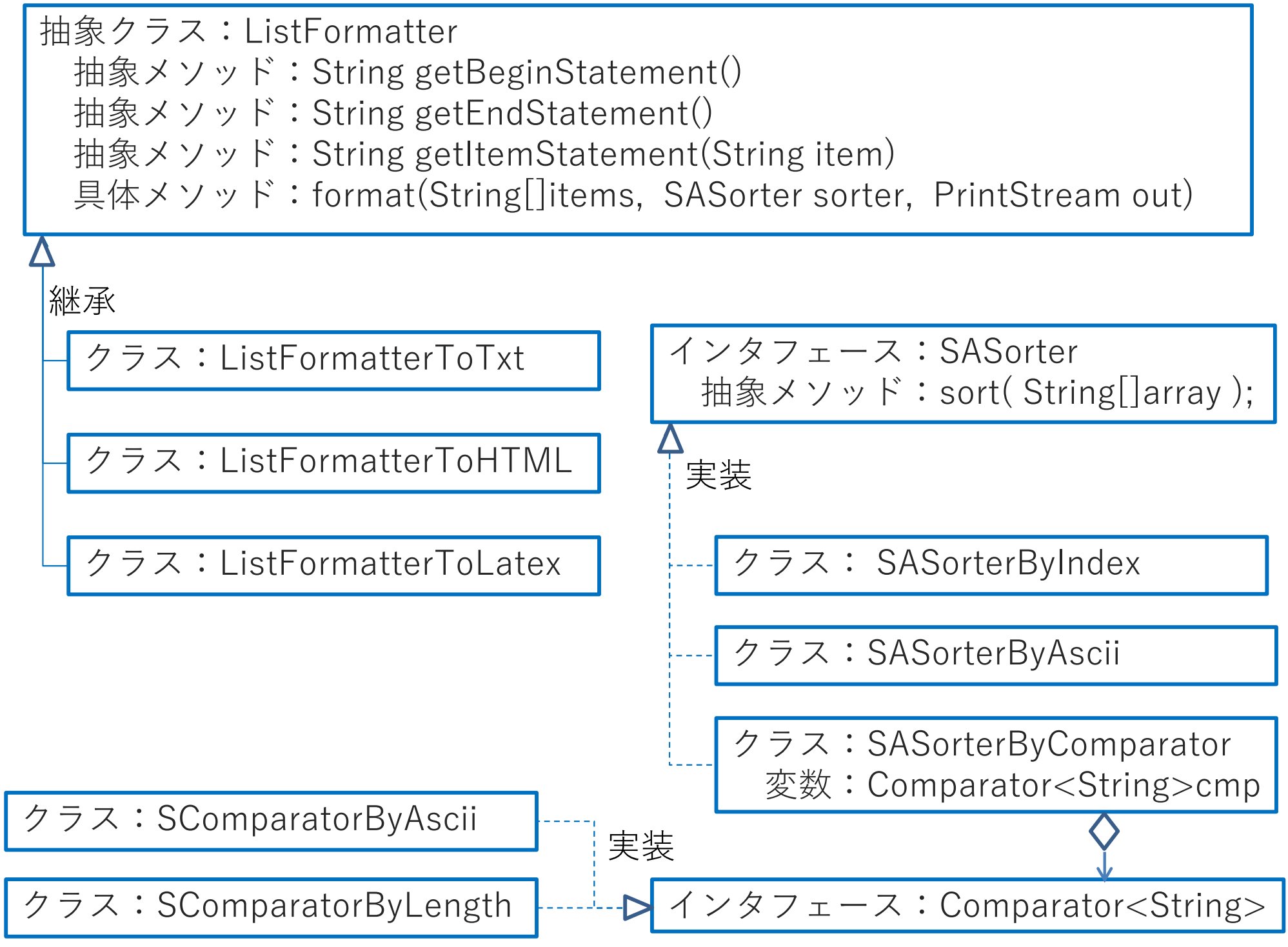
- どの様なプログラムが考えられるか？
  - ソート基準：３種（ソートなし，辞書順，文字列長昇順）
  - 出力先：２種（画面，ファイル）
  - 出力形式：３種（テキスト，HTML，Latex）
  - 3基準 x 2出力先 x 3形式 = 18個のプログラムを作る？
    - ソート基準が一つ増える度に，2出力先 x 3形式のプログラムを作る？
    - 出力形式が一つ増える度に，3ソート基準 x 2出力先のプログラムを作る？

- 望ましいプログラム
  - アルゴリズムの再利用が可能
    - 基本アルゴリズム（「配列のコピー」「並べ替え」「ヘッダーの出力」「各項目の出力」「フッターの出力」）は同じなので，この部分は一回作るだけにしたい
  - 出力先の指定が可能
    - プログラム実行時に「出力先」が指定出来れば，実行部分（main部）だけを変更すればよいだけになるはず
  - 形式の追加と保守が容易
    - 新たな形式が必要となったときは「差分だけを追加」したい（拡張性）
    - 既にあるプログラムを改変したくない（保守性）

- オブジェクト指向の考え方を用い，これらの要望を実現する

関係図

lec01/*.java



リストの整列・表示する主クラス

抽象クラスとして準備

```
/* リストを整列し出力するクラス */
abstract class ListFormatter{
    abstract String getBeginStatement(); //リストの開始文字列
    abstract String getEndStatement(); //リストの終了文字列
    abstract String getItemStatement(String item); //各要素

    //文字列配列を出力するメソッド
    void format(String[]items, SASorter sorter, PrintStream out) {
        //配列をコピーする
        String[]copy = new String[ items.length ];
        System.arraycopy(items, 0, copy, 0, items.length);

        sorter.sort( copy ); //コピーした配列をソートする
                ソートに関しては後述
        out.println( getBeginStatement() ); //ヘッダーの出力
        for(int i = 0; i < copy.length; i++) {
            out.println( getItemStatement( copy[i]) ); //各項目の出力
        }
        out.println( getEndStatement() ); //フッターの出力
    }
}
```

種々の形式に対応するため，
　抽象メソッドとし，
継承先（サブクラス）で，
　具体的な実装を行う

整列を実行するオブジェクト

出力先オブジェクト

配列コピー：arraycopy(コピー元配列，コピー元開始添字，コピー先配列，コピー先開始添字，コピー要素数)

出力までのアルゴリズムは同じ
　配列コピー
　コピー配列をソート／出力

抽象クラス内の具体メソッド内で，
　抽象メソッドを呼び出す
　（Template Method パターン）

“実行時”に，sorter と out を指定することで，種々の整列基準・出力先に対応する

抽象メソッドと抽象クラス

- 抽象メソッド
  - 実装を持たず，メソッドの規約（型，引数，意味）だけを定義したメソッド
  - メソッド定義時に先頭にabstractを付ける．本体の記述はない
    - `abstract String getBeginStatement();`
    - `abstract String getEndStatement();`
    - `abstract String getItemStatement(String item);`

- メソッドの規約？？中身がない？
  - 別の例：図形を表すオブジェクトが（自身の）面積を返すメソッド
    - `abstract float getArea( );`
    - メソッド名・引数・「面積を返す」という意味は決まっているが，
      具体的な図形が決まらないと，計算方法が決まらず，実装できない

- 抽象クラス：抽象メソッドを含むクラス
  - 抽象クラスのインスタンスを生成することはできない．
  - 継承先の（具体）クラスでは，抽象メソッドを実装しなければならない
    - オーバーライド（メソッドの上書き）忘れの防止

抽象クラス（ListFormatter）の継承

```java
/* HTML形式で出力するクラス */
class ListFormatterToHTML extends ListFormatter {
    @Override
    String getBeginStatement() {
        return "<ul>";
    }
    @Override
    String getEndStatement() {
        return "</ul>";
    }
    @Override
    String getItemStatement(String item) {
        return "<li>"+item+"</li>";
    }
}
```

```java
/* Latex形式で出力するクラス */
class ListFormatterToLatex extends ListFormatter {
    @Override
    String getBeginStatement() {
        return "\\begin{itemize}";
    }
    @Override
    String getEndStatement() {
        return "\\end{itemize}";
    }
    @Override
    String getItemStatement(String item) {
        return "\\item "+item;
    }
}
```

- 継承：　class クラス名 extends 親クラス名{ .. }
  - Javaでは多重継承はできない
  - コンストラクタは継承されない
- 親クラスで定義された変数・メソッドは（カプセル化による制限はあるが）利用可能
  - 差分プログラミング：親クラスとの差分だけをプログラムする
- オーバーライド：親クラスで定義されたメソッドの上書き
  - 子クラス独自の実装を実現
  - （具体クラスでは）抽象メソッドの実装は必須

並べ替え基準の実装

```
interface SASorter {
    // (ある基準に基づいて) arrayをソートする
    void sort( String[]array );
}
```

インタフェース内のメソッドは抽象メソッド

```
/* 与えられた順番のままソートする (=何もしない) */
class SASorterByIndex implements SASorter {
    @Override
    public void sort(String[] array) {
        //do nothing
    }
}

/* 自作の単純ソートを用い，辞書順にソートする */
class SASorterByAscii implements SASorter {
    @Override
    public void sort(String[] array) {
        for(int i = 0; i < array.length-1; i++) {
            for(int j = i+1; j < array.length; j++) {
                if(array[i].compareTo(array[j]) > 0) {
                    String tmp = array[i]; array[i] = array[j]; array[j] = tmp;
                }
            }
        }
    }
}
```

インタフェースを利用する
sort(array)メソッドでarrayをソート

インタフェースを実装したクラスを準備する
このクラスのインスタンスを，
ListFormatter#format に渡す

- SASortByIndexのインスタンスを渡せばソートしない
- SASortByAsciiのインスタンスを渡せば辞書順に整列

単純ソートによる並び替え

文字列X, Y の比較はX.compareTo(Y) で行う
X < Y の場合, X.compareTo(Y) < 0
X == Y の場合, X.compareTo(Y) == 0
X > Y の場合, X.compareTo(Y) > 0

インタフェース

- インタフェースとは
  - 抽象メソッド（と定数）の集合．メソッド構成に対する名前（目印）
  - 該当クラスが備えているべきメソッド（has_a関係）を定義するもの
  - クラス階層（is_a関係）とは異なる「型」（種別）を与える

- インタフェースの宣言： interface インタフェース名 { }
- インタフェースを実装したクラスの宣言
  - class クラス名 implements インタフェース名
  - インタフェース = 抽象メソッドの集合

    → implementsしたクラスは，抽象メソッドを実装する必要がある

- クラス階層とは独立（クラスとは独立した型階層を定義できる）
  - SASorterを実装した２つのクラスSASorterByIndexとSASorterByAscii間に継承関係なし
  - 両者を SASorter型 のオブジェクトとして利用することができる．
  - ListFormatter#format(String[]items, SASorter sorter, PrintStream out) において，sorterの型はSASorter
    - SASorterByIndex型， SASorterByAscii型のオブジェクトは，SASorter型の変数に代入可能
      - 多態性：一つのオブジェクトが複数の型を持っているようにふるまう

Comparator インタフェースと並び替え

- Javaでは，並べ替えを行うメソッドが準備されている
  - java.util.Arrays.sort(配列，並べ替え基準)

- インタフェース Comparator<String>を実装したオブジェクトを指定することで，種々の基準での並べ替えを実現している
  - 「インタフェース」を実装していれば，クラスは何でもOK

SASorterを実装
SASorterByIndexやSASorterByAsciiと
同様に扱うことが可能

```
/* 基準 (Comparator)を与え，ライブラリを用いてソートする */
class SASorterByComparator implements SASorter {
    Comparator<String>cmp;//並べ替えの基準
    SASorterByComparator(Comparator<String>comparator){
        cmp = comparator;
    }
    @Override
    public void sort(String[] array) {
        java.util.Arrays.sort(array, cmp);
    }
}
```

コンストラクタを用い，
オブジェクト生成時に
並べ替え基準comparatorを渡す

```
SASorter sorter =
    new SASorterByComparator(
        new SComparatorByLength()
    );
```

cmpを指定する（渡す）ことで，
ソート自体は，Javaにやらせる（一皮かぶせるAdapterパターン）

Comparator<String>を実装したクラス

```java
//ソートの基準（辞書順）
class SComparatorByAscii implements Comparator<String>{
    @Override
    public int compare(String arg0, String arg1) {
        return arg0.compareTo(arg1);
    }
}
```

Comparator<String>インタフェースを実装

arg0.compareTo(arg1) …
文字列 arg0 と arg1を比較する
(see below)

```java
//ソートの基準（1. 文字列長，2.辞書順）
class SComparatorByLength implements Comparator<String>{
    @Override
    public int compare(String arg0, String arg1) {
        int cmp = arg0.length() - arg1.length();
        if(cmp == 0) {//文字列長が同じなら，コードで比較する
            cmp = arg0.compareTo(arg1);
        }
        return cmp;
    }
}
```

- Comparator<String>を実装： implements Comparator<String>
- public メソッド int compare(String arg0, String arg1)の実装が必要
  - 基本的な考え方（compareToと同じ考え方， arg0 – arg1 ）
    - arg0 < arg1 のときは　負値を返す
    - arg0 == arg1 のときは 0 値0を返す
    - arg0 > arg1 のときは　正値を返す
  - 文字列同士の比較には，文字列1.compareTo(文字列2)を利用する

- SASorter sorter = new SASorterByComparator(new SComparatorByLength());

補足：総称型（ジェネリクス）

- ジェネリクス：様々な型に対応する汎用的なクラスやメソッドを作る機能
  - String型のインスタンスを入れるリストも，Integer型のインスタンスを入れるリストも，リストとしての機能は同じ
- クラス・インタフェースを定義する際に，<>で囲む

  総称型のまま利用する場合の例
  - class MyClass<T> { … }　　//TやEは総称型
  - interface MyInt <E> { … }
  - class MySubClass<T,E> extends MyClass<T> implements MyInt<E> { … }

  具体的な型を指定する場合の例
  - class MySubClass extends MyClass<String> implements MyInt<Integer> { … }
- インスタンスの生成
  - 総称型のままではNG
  - <>内に，具体的な型を指定する
- 補足：難しく考えすぎないでOK．授業では，必要最小限の利用に留めます
  - 要は，T型やE型など，実際には存在しない型の指定が可能ということ
  - （インスタンスの生成など）実際の利用時に，具体的な型を指定する

出力先の指定

- 今回は，PrintStream クラスのオブジェクトを渡すことにする

- 標準出力に出力
  - PrintStream out = System.out;

- ファイルに出力
  - PrintStream out = new PrintStream(new File(ファイル名));

- 実行時に，PrintStream型のインスタンス out を

  ListFormatter#format(String[]items, SortStringArray sorter, PrintStream out)

  に渡すことで，出力先を制御する

プログラムを実行するクラス

public static void main(String[]args) から開始
static: 静的＝クラスに属する
→ インスタンス生成の必要なし

throws exception：
例外処理（未学修内容）

```
public class _Sample01Main {
    public static void main(String[]args) throws Exception {
        String[]items = { "abcd", "xy", "123" }; //対象とする文字列の配列

        SASorter [] sorters = {//整列基準の準備
                new SASorterByIndex(),
                new SASorterByAscii(),
                new SASorterByComparator(new SComparatorByAscii()),
                new SASorterByComparator(new SComparatorByLength()),
        };
        ListFormatter [] formatters = { //出力形式
            new ListFormatterToTxt(),
            new ListFormatterToHTML(),
            new ListFormatterToLatex(),
        };
        PrintStream out = System.out; //出力先
        //PrintStream out = new PrintStream(new File("lec01/out.txt"));

        for(int i = 0; i < formatters.length; i++) { //種々の組み合わせで出力する
            for(int j = 0; j < sorters.length; j++) {
                formatters[i].format(items, sorters[j], out); //実際の出力
                out.println();
            }
        }
        out.close();
    }
}
```

SASorterByIndex,
SASorterByAscii
SASorterByComparator
はバラバラのクラスだが,
すべて同一インタフェース
SASorterを実装しているので,
SASorter型として扱える

ListFormatterToTxt, ListFormatterToHTML,
ListFormatterToLatexは,
すべてListFormatterの子クラス

形式

整列基準

出力先

コンパイル方法: javac lec01/_Sample01Main.java
実行方法: java lec01/_Sample01Main

ここまでを振り返って

オブジェクト指向らしさを振り返る.
保守性・再利用性・拡張性に着目しよう
三大要素：継承・多態性・カプセル化

- 実行部（mainメソッド）と部品の分離
- 抽象クラスの利用
  - 抽象メソッドを実装した（具体）クラスが必要
  - 異なる「実装」を持つインスタンスを統一的に扱うことが可能
    - abstract class ListFormatter
      - class ListFormatterToTxt extends ListFormatter
      - class ListFormatterToHTML extends ListFormatter
      - class ListFormatterToLatex extends ListFormatter
  - 共通部分（アルゴリズム）は共通化されている（再利用性）
- インタフェースの利用
  - クラスを超えた「規格」の統一
    - Interface SASorter
      - Class SASorterByIndex implements SASorter
      - Class SASorterByAscii implement SASorter
      - Class SASorterByComparator implements SASorter
- 多態性
  - インスタンスが持つメソッドが動く
  - 部品＝インスタンスを変えることで動作が変わる（拡張性）

演習 1

- 動かしてみよう1
  - コンパイル方法： javac lec01/_Sample01Main.java
  - 実行方法：java lec01/_Sample01Main.java
- 動かしてみよう２：ファイルに出力するよう，プログラムを修正してみよう

- どんなクラスを作成すべきか熟慮した上で，下記を実現するプログラム（クラス）を作成しなさい
  - ディレクトリ lec01内に作成しましょう
  - package宣言に注意しよう.
    - 各ファイル(.java)の先頭に package lec01; が必要

- 作成してみよう1：「文字列長の降順」に出力する
- 作成してみよう2：HTMLの Ordered List形式で出力する
- 作成してみよう3：Latexの enumerate環境で出力する
- 作成してみよう4：テキストで 1, 2, ...と順序をいれて出力する
  - 右図の様に出力してください

```
-----
1. abcd
2. xy
3. 123
-----
```

- 作成したプログラムをBlackBoardから提出してください
  - 指定ファイルの本文に，プログラム内容（テキスト）を貼り付ける（コピペする）形で提出してください

※指定ファイル（In-class_Report.txt）中の問１は解答不要です

デザインパターン

デザインパターン

- 「デザインパターン」（see Wikipedia）
  - 典型的な状況（パターン）における，クラス・インタフェースの構成に関する過去の知見を集約したもの
  - 万能ではないが，複雑なシステムを作成する際の一つの指針となる
- デザインパターンを学ぶ主目的
  - 再利用しやすいプログラム構成（クラス・インタフェースの構成）の修得
- the Gang of Fourによるデザインパターン
  - the Gang of Four（GoF）：E. Gamma, R. Helm, R. Johnson, J. Vlissiders
  - 23個のデザインパターンを定義

- 生成に関するパターン：
  1. Abstract Factory
  2. Builder
  3. Factory Method
  4. Prototype
  5. Singleton
- 構造に関するパターン：
  6. Adapter
  7. Bridge
  8. Composite
  9. Decorator
  10. Façade
  11. Flyweight
  12. Proxy
- 振る舞いに関するパターン：
  13. Chain of Responsibility
  14. Command
  15. Interpreter
  16. Iterator
  17. Mediator
  18. Memento
  19. Observer
  20. State
  21. Strategy
  22. Template Method
  23. Visitor

デザインパターン Q&A

Q：GoFのデザインパターンは23個もあり，全部覚えるのは難しいです．

A：無理に全部を覚える必要はありません．GoFが提唱しているデザインパターンがすべて同じような頻度で使われているわけではないからです．特に，機械的に丸暗記してしまうのはあまり意味がありません．それよりも，そのデザインパターンが問題をどのように解決しているかを自分の頭で理解することが大切です．

Q：デザインパタンと言われている解法は，すべて当たり前のことばかりです．注目したり，改めて勉強する価値があるとは思えません．どうしてデザインパターンが重要なのですか

A：たくさんの経験を積んだプログラマは，デザインパターンとして紹介されているものを「当たり前」だと感じることが多いようです．それは当然のことです．もともとデザインパターンは，プログラマが繰り返し経験してきた問題とそれに対する解法をまとめたものだからです．デザインパターンが重要なのは，経験を積んだプログラマの知恵を効率よく学ぶことができるからです．

結城浩 『Java言語で学ぶデザインパターン入門第３版』 SBクリエイティブ(2021)より

デザインパターン Q&A

Q：デザインパターンは初級プログラマにも役立ちますか

A：もちろんです．プログラミング言語の基本を学んで，自分で少しずつ
プログラムを組めるようになった人が，デザインパターンを通して
「オブジェクト指向プログラミングでは，どの様な点に注意してプログラムを
作るのか」を学ぶのは良いことです．また，クラスライブラリを自分で「使う」
場合にも，デザインパターンの知識は役に立ちます．なぜなら，クラス
ライブラリの多くの部分が，デザインパターンに関連しているからです．

デザインパターンを学ぶ前に：デザインパターンを理解するためのヒント

- デザインパターンはクラスライブラリそのものではない
  - デザインパターンは，クラスライブラリよりも一般的な概念
  - クラスライブラリは部品となったプログラムそのもの
  - デザインパターンは，部品がどのように組み立てられているか，個々の部品がどの様に関連して大きな機能を果たすのかを表現したもの
- プログラムを完成品として見ないこと
  - デザインパターンの目標の一つは，プログラムを再利用可能にすること
  - どうやってプログラムを「部品」として再利用するかを考える
  - プログラム例を「完成品」として見るのではなく，今後「機能を拡張していくもの」「変更を加えていくもの」としてみる
    - どの様な機能が拡張される可能性があるか？
    - その機能拡張を行うとき，修正が必要・不要なのはどのクラスか？
- 図は見るのではなく，読むこと
  - クラス図等を「絵」として眺めるのではなく，図中の構成要素が何を意味しているのかを丁寧に一つ一つ追いかけること
- 自分で例を考えること
  - 単にプログラム例を読むのではなく，自分なりの例を考えてみましょう．
- 役割を理解すること
  - デザインパターンごとに，クラス・インタエースにはそれぞれに役割が割り振られる．それぞれの役割を理解する

結城浩 『Java言語で学ぶデザインパターン入門第３版』 SBクリエイティブ(2021)より